

When Tool Errors Survive but Recovery Records Do Not: Auditing Agent Compensation Interfaces

Working manuscript, incomplete. The results below are executed software probes and an archival artifact audit. They are not model-task performance results, estimates of deployed failure prevalence or an independently validated new recovery algorithm. The evidence is not yet sufficient for submission.

Abstract

Agent recovery components must distinguish tool execution, reported status and persisted effects. We audit these agreements in a pinned release of RobustAgentCompensation (RAC), using controlled native integrations and a separate audit of its published artifact. Normally returned error messages can coexist with completed recovery records, and a schema-declared compensation pair is not automatically discovered. Matched MCP adapter versions isolate an error-delivery difference, while independent agent-saga and SagaLLM controls delimit transport and coordinator behavior. Twenty-five native airline cases distinguish failed effects from errors after completed cancellation. A census of 2,000 reservations shows that repeated cancellation preserves a narrow status/net contract while changing ledger history. In a second native environment, all 423 direct retail cancellations satisfy the declared accounting contract, whereas 120 payment-change/cancellation paths across 102 orders return normally with cancelled status but unbalanced per-method ledgers. An independent offline verifier recomputes the saved states and accounting results. Separately, all 280 JSON files in the released RAC archive are audited for duplicate records and explicit structured-status contradictions; none of the latter are observed under the fixed screen. These findings identify reproducible integration and composition failures, without attributing constructed failures to the published benchmark. The experiments contain no autonomous model decisions and establish neither framework-wide rankings, deployed failure prevalence nor external financial harm. They support explicit effect contracts and versioned recovery evidence, not a new general recovery algorithm.

1. Introduction

Recovering from an unsuccessful agent action can require more than retrying the failed call. Earlier actions may have modified external state and need compensation. A recovery manager typically records those actions, maps them to compensating tools, executes the compensations and updates its log. Each step introduces an agreement between components: what is recorded, which result constitutes failure, and what a successful compensation establishes.

Tool libraries represent failures in several ways. A function can raise an exception, return a typed error message, or place an application-defined error inside ordinary content. An adapter may deliberately turn an exception into an observation that allows an agent to continue. That transformation is reasonable for a conversational loop, but a downstream recovery component must interpret the resulting value consistently. It cannot assume that every normal return means the requested state transition occurred.

This study investigates concrete implementations rather than proposing a new general transaction protocol. Its primary subject is the public RAC code and its archived artifact, with selected independent implementation controls. We ask which failures can be demonstrated through actual component execution, which depend on the selected dependency version, and whether the released experimental traces independently show those failures. Separating these questions prevents a unit-level counterexample from becoming an unsupported claim about benchmark performance or deployed reliability.

2. Observational distinctions

We record four separate properties of an operation: whether invocation raises, what error status its returned object carries, how the recovery log classifies the operation, and whether the fixture's intended state change occurred. These observations need not agree. A normal return may carry an explicit error; a positive status may accompany a silent no-op; an empty log may coexist with a prior untracked effect.

Our fixture uses a single active-booking flag. A successful forward operation sets it, and its specified compensation clears it. This makes residual effect observable independently of the recovery report. It is a narrow local contract, not a demand that real compensating transactions restore every historical database byte. In particular, the absence of recorded actions establishes only that the manager has an empty plan. It does not certify that no external effect exists.

We reserve the phrase reported successful rollback with residual effect for a run that actually attempts the declared compensation, reports success and leaves the flag active. We describe failed automatic discovery separately. The latter returns an accurate description of its empty recorded plan, while failing to track the workflow's schema-declared compensatable operation.

3. Subject, versions and experimental method

The current RAC source is fixed at commit 068c327ab3f9b79a86a39503946e26320b74b97a. Its source blobs are checked against the pinned Git tree before import. The published Zenodo archive is separately verified by its released MD5, a recorded SHA-256 and ZIP integrity checks. Archived Python files are extracted without modification and tested separately. No probe rewrites the third-party implementation.

The real LangChain experiments use `langchain-core==1.6.3`. Native MCP tests add `mcp==1.30.0`, `langchain-mcp-adapters==0.3.2` and `pydantic==2.13.5` in an isolated environment. A further comparison changes only the adapter to 0.2.2 through an isolated import overlay. These are declared experimental configurations, not reconstructions of the authors' original dependency set. The archive does not supply a complete dependency lock for that reconstruction.

All tests use local constructed tools, without a model or a live booking service. Connection guards prohibit external network access. The native MCP fixture uses actual stdio subprocesses, SDK initialization, schema exchange and tool invocation. Its state file persists across the adapter's per-call server sessions. Windows event-loop socket pairs are recorded separately from MCP transport. The parent sees no external connection attempts.

The probe series covers distinct execution paths, not independent random samples. Twelve initial core/adapter cases isolate returned status; 24 cases exercise retry and alternative-action interceptors; 21 use actual LangChain StructuredTool/ToolMessage objects; eight additional controls isolate invocation format. Fourteen archived-source rollback cases address source-version scope. The native MCP study contains twelve execution controls and one unmodified automatic-discovery workflow, followed by six matched older-adapter executions. We do not sum these numbers into a task-level sample size or a defect rate.

Independent controls pin agent-saga commit 4310ff570e60c42c081ae216e87a1ccb093525d4 and SagaLLM commit 2781c33edc4005b066671d5dc3cad157fae14f11. Downloaded source blobs are verified against their Git trees. Eight agent-saga controls use its unmodified `upstreamServer` and `sagaMCPProxy` with the existing SDK fixture, a local write-ahead log, an explicitly supplied compensation policy and rollback boundary. The outer client-facing stdio loop is not exercised. Six SagaLLM controls execute its unmodified coordinator with scripted agents, not its model-driven agent. The isolated parent environment adds `colorama 0.4.6`, `graphviz 0.21` and `pydantic 2.13.5`; the child uses the existing MCP environment. These fixtures were selected after the RAC diagnosis, so their selection is not independent of the observed mechanism.

4. Returned errors and rollback records

The core rollback path calls the compensation executor and immediately marks the original action compensated. It does not inspect the returned action result. Raised exceptions instead reach its failure branch. The author-provided LangChain executor catches a tool exception and returns an error action result. Composing these two components therefore changes an exposed failure into a reported successful rollback, with the fixture's booking still active.

Supplying the implementation's explicit error-detection strategy does not change this rollback branch. A diagnostic executor that rejects its documented error result prevents the false success for explicit failures, but cannot detect a silent no-op that returns a positive status. This control separates an interface mismatch from a

missing postcondition observation. Neither status checking nor postcondition checking is claimed as a new method.

The archived-source tests reproduce the ignored-result mechanism in the published code, using fourteen selected combinations of executor and tool behavior. Direct exceptions are exposed, whereas an adapter-produced error result can be ignored. The dependency environment remains current, so this is evidence about execution of the archived branch under the stated setup; it is not evidence that a corresponding error occurred in a published trial.

5. Native MCP boundaries

5.1 Automatic pair discovery

The fixture declares a `book -> cancel` relation through the input-schema extension documented by RAC. That extension survives in the converted tool's `args_schema`, and RAC's own parser recognizes the dictionary when called directly. Its high-level client nevertheless discovers no pair in the tested configuration.

The source path explains the mismatch. Raw-schema discovery expects an older client's `_sessions` attribute. The fallback obtains `get_input_schema()`, which returns a Pydantic model class here, while the pair parser expects a dictionary. Returning that value also bypasses the later `args_schema` branch.

In the unmodified-factory workflow, a successful booking sets the flag but the wrapper records no action. Rollback then reports an empty plan and attempts no cancellation. This observation concerns one documented declaration form and these dependencies. It does not show that every metadata form or every application configuration fails.

5.2 Error status present in the returned message

Twelve further native executions explicitly register the compensation pair using the manager's public method. This disclosed diagnostic intervention isolates execution semantics after discovery; it is not counted as successful automatic discovery. The client, wrapper, executor and recovery manager remain unchanged.

Constructed behavior	Default adapter	Exception-raising diagnostic
Successful cancellation	Flag cleared; compensated	Same
Cancellation returns MCP execution error	Flag active; compensated	CriticalFailure; not compensated
Cancellation handler raises	Flag active; compensated	CriticalFailure; not compensated
Cancellation returns success without effect	Flag active; compensated	Same
Forward error through plain arguments	Content list; log COMPLETED	Raised error; log FAILED
Forward error through full tool-call envelope	Error ToolMessage; log COMPLETED	Raised error; log FAILED

In the last row, the returned ToolMessage explicitly retains its error status. The wrapper still marks the action completed because its result check handles strings and dictionaries, but not that message type. Plain-argument invocation returns content blocks instead, which the same predicate also does not handle. The rollback path's ignored normal return supplies the second discrepancy. The MCP SDK converts a server-handler exception into an execution-error response; this case does not simulate a transport outage.

5.3 A controlled package-version comparison

The diagnostic exception mode requires a raw-client replacement because the RAC factory does not expose the adapter's error-handling flag. We therefore also execute six matched cases with adapter version 0.2.2, leaving the other packages, server and RAC source fixed. Under the older adapter's default, both cancellation errors raise `CriticalFailure` and both forward errors produce `FAILED` records. Successful cancellation and the silent no-op behave as before. Automatic pair discovery remains unsuccessful.

This comparison isolates a package-version effect in the constructed workflow. It does not establish which adapter version the RAC paper used. In particular, the remaining packages are still current, and the newer default was released after the paper. Attributing this MCP-specific behavior retrospectively to the original reported model experiments would be unwarranted.

5.4 Independent native MCP transport control

`agent-saga` provides a useful positive control. Its stdio transport raises on an MCP `isError` result. Using exactly the same fixture script and four rollback behaviors as the RAC default-adapter study produces the following observations. The validation checks raw fixture-state equality across implementations and consistency between `agent-saga`'s report and persisted log.

Compensation behavior	Residual booking	RAC default reports success	agent-saga reports clean
Successful cancellation	No	Yes	Yes
Explicit MCP error	Yes	Yes	No
Server exception serialized as MCP error	Yes	Yes	No
Successful response with no effect	Yes	Yes	Yes

The two explicit errors become `COMPENSATION_FAILED` in `agent-saga`. In its forward-error controls, the operation is recorded as `UNKNOWN`, and the explicit rollback uses the booking identifier supplied in the original arguments. This conservatively preserves the possibility of an effect even though the constructed failing forward operation makes none. A silent no-op still produces a positive rollback report with a residual booking. Thus correct error propagation addresses one boundary without proving an external postcondition. The comparison does not test crash recovery, connector services, automatic pair discovery or overall framework reliability.

5.5 An exception-based coordinator interface

The independent `SagaLLM coordinator` calls rollback on agents whose forward calls returned before a subsequent exception. With scripted agents, a raised compensation error produces a warning; a normally returned error dictionary and a silent no-op both produce a printed rollback-completion message while leaving the local effect active. The dictionary is an experimenter-defined value, not an advertised `SagaLLM` error protocol. This result demonstrates its exception convention and should not be presented as violation of a declared typed-result contract.

Two further controls raise before and after the forward effect. In both, the currently failing agent is absent from the coordinator's completed list and its compensation is not attempted; only the after-effect case leaves a residual booking. These are scripted coordinator paths, not a reproduction of `SagaLLM`'s model benchmark. The contrast with `agent-saga`'s explicit `UNKNOWN` state illustrates why a failed return and an absent effect are different facts. This distinction is established in prior recovery work and is not a new algorithmic contribution here.

5.6 Native reservation and payment-ledger state

The preceding binary booking fixture makes transport behavior inspectable, but does not exercise a business tool's coupled state. We therefore add unchanged `tau2 airline tools` on the official unit-test database. One input is its single-flight/card-payment fixture; the other is an explicitly constructed two-flight extension paid with a certificate and a card. Source hashes and fixture bytes are pinned. These are selected workflow controls, not sampled official tasks or an evaluation of an autonomous airline agent.

We define the cancellation contract as cancelled reservation status and zero net amount in the reservation payment ledger. Full database equality is unsuitable: cancellation retains the reservation and refund entries, and the pinned implementation explicitly omits seat release. We report inventory and payment-instrument restoration separately without counting these omitted effects as violations of our narrower declared contract.

Actual stdio MCP exposes the native tool schemas and methods. Four imposed conditions locate the failure before or after the relevant effect: normal cancellation; a native invalid-ID exception; cancellation on a disposable database copy; and an acknowledgement exception after cancellation of the actual database. Equal native return payloads are serialized for the normal and copy-only conditions. We compare RAC's default adapter, its diagnostic raising configuration, and agent-saga, giving 24 executions across two inputs.

The result ID is obtained from the actual booking output. RAC's stored result wraps this adapter's content list in a Python-literal raw string; the main matrix therefore explicitly supplies a bounded decoder in the developer mapper. agent-saga uses its configured structured-content path. A twenty-fifth control omits the RAC stored-result adaptation: it reports COMPENSATED/success, but no native cancellation invocation occurs and the reservation remains uncanceled. The adapted normal control completes cancellation. This contrast exposes a result-storage boundary without crediting our adaptation as automatic parameter extraction.

Both booking profiles produce the following outcomes:

Imposed cancellation condition	RAC default	RAC raising control	agent-saga	Native cancellation contract
Normal	Success	Success	Clean	Satisfied
Native error before effect	Success	Raises CriticalFailure	Incomplete	Unsatisfied
Disposable-copy effect	Success	Success	Clean	Unsatisfied
Error after actual effect	Success	Raises CriticalFailure	Incomplete	Satisfied

RAC's raising condition leaves the action COMPLETED and supplies no returned rollback report; we retain the exception and do not turn absence of a report into a measured boolean. agent-saga records COMPENSATION_FAILED on the error conditions. Failure after the actual effect illustrates why an error flag cannot establish that compensation did not occur; conservative uncertainty is not classified as a defect.

Independent state recomputation within our research process validates all 25 saved databases and eight WAL files. Each of the eight input/fault groups has exactly equal final database states across clients. For six matched input/client normal-versus-copy comparisons, native cancellation payloads are equal while persisted database states differ. Thus these controls separate correct error transport from verification of the requested effect. They establish neither an error prevalence estimate nor a novel general isolation or postcondition-verification algorithm. Both partial setup attempts and their causes are retained, and repeated cases are not counted as extra independent observations.

5.7 Contract-preserving retries can still change recorded state

An acknowledgement error after cancellation raises a separate question: what changes if the same cancellation is repeated? We register a direct-call census of every reservation in the pinned public airline database before execution. The population contains 2,000 reservations, all initially having one payment-history entry and unset status: 965 gift-card payments and 1,035 credit-card payments. This is a finite input population with homogeneous ledger structure, not 2,000 independent agent tasks or a deployed workload.

For each reservation, we restore its initial value before each of three diagnostic policies. The first cancels once. The second cancels three times, with an experimenter-imposed acknowledgement exception after each native return. The third cancels once with the same imposed exception, then calls native `get_reservation_details` and stops if cancelled status and zero signed ledger net are observed. These calls execute unchanged native methods directly; they do not pass through MCP or a recovery framework. We do not attribute the chosen retry behavior to RAC or agent-saga.

Policy	Cancels / reads per reservation	Final ledger entries	Cancellation contract satisfied	State equal to single cancellation
Single cancellation	1 / 0	2	2,000 / 2,000	Reference
Three cancellation attempts	3 / 0	8	2,000 / 2,000	0 / 2,000
Authoritative read before retry	1 / 1	2	2,000 / 2,000	2,000 / 2,000

The native method appends the negation of every existing payment-history entry, including previously appended negative entries. The successive ledger sizes are therefore 2, 4 and 8 for every reservation, while all 10,000 observed post-cancellation states retain zero signed net and cancelled status. Strict target-state equality fails under repeated cancellation even though the narrow contract remains satisfied. Increased gross ledger entries do not establish duplicate external refunds or financial loss: this tool records ledger values and does not execute an external payment. Nor does its interface promise strict state idempotence. These observations delimit the consequence rather than labeling every repeated write a failed recovery.

The read-based diagnostic produces exactly the same target state as single cancellation for all records, with one extra read and two fewer cancellation calls than the three-attempt control. Its observer is authoritative and synchronous by construction. This is an upper-bound diagnostic, not a novel policy or evidence for reliability under stale reads or concurrent writers. A separate validator computes append-negation transitions directly from the original database, without importing the tool implementation or measurement function. It validates every saved target field and all ledger nets. The initial shared database hash is restored after all per-record resets. A ZIP path-handling correction is retained separately from the unchanged raw run; it does not create additional observations.

5.8 Normal returns can conceal a composition-level ledger failure

The previous controls distinguish transport errors from completed effects. A further diagnostic asks whether even two ordinary, normally returning native calls can leave a cancelled order outside an explicit accounting contract. We use the unchanged [retail tools](#) and complete public synthetic retail database at the same tau2 revision. Its policy allows a payment-method change to retain pending status and allows pending orders to be cancelled. This probe assumes authorization for both operations; it does not evaluate an agent's authentication or dialogue.

The protocol was frozen after source inspection and before mutation. That inspection already suggested the mechanism: payment change appends a new payment and an original-method refund, while cancellation subsequently appends a refund for every existing ledger entry, including refunds. Thus the experiment is a source-derived composition diagnostic, not a blind discovery or estimate of failures in realistic customer conversations.

All 423 initially pending orders have one positive initial payment and a known payment method, satisfying the fixed eligibility rules. The other 577 orders are excluded for their initial status. We execute one direct cancellation

for each eligible order, independently resetting the affected state. Of those orders, 102 admit at least one different stored payment method under the native preconditions. All 120 eligible order/alternative pairs receive payment change followed by cancellation from the initial state. These pairs are not 120 independent initial orders.

The cancellation contract requires cancelled status, zero signed ledger net for each payment method, and the gift-card balances implied by refunding the original payment exactly once. Payments and refunds have opposite signs; an aggregate net alone is insufficient because opposite errors on different instruments could cancel. The contract does not require erasing payment history or proving an external refund.

Native path	Paths	Normal return and cancelled status	Ledger/balance contract met
Direct cancellation	423	423	423
Change payment, then cancel	120	120	0

The 663 native calls raise no exceptions. After each payment change, the old method's signed net is zero and the new method carries the original amount A . After cancellation, the new method's net is zero, but the old method's net is minus $2A$. Every composed path has six ledger entries, compared with two in its direct-cancellation control. The 59 compositions starting with a gift card also leave its balance $2A$ above the declared cancellation target. The remaining 61 compositions still fail the ledger component. We observed all six eligible original/new payment-type pairings; no favorable pair was selected for the reported counts.

An independent offline verifier recomputes eligibility, every transition, per-method net, and gift-card balance from the original database and saved records without importing the native tools or original measurement function. The runtime also verifies restoration of the full model database after all per-path resets. This census contains no injected acknowledgement faults, MCP adapters, compensation framework or autonomous model decisions. It shows why a status-only recovery check is insufficient in this native composition; it does not attribute the behavior to RAC or revise a published benchmark success rate. Synthetic ledger imbalances are not evidence of real financial loss. The selected sequence and single pinned implementation limit its scope.

6. What the released experiment archive shows

We separately inspect all 280 JSON files in the published archive. The audit distinguishes execution traces from aggregate records and progress snapshots. The 13 aggregate JSON files contain 1,272 record occurrences but only 224 distinct canonical records. Thirty-five REALM snapshots share one start time and form an exact event-prefix chain. Neither group can be pooled as independent trials merely by counting files or rows.

A conservative screen asks whether a recorded successful event has an immediate structured result carrying an explicit contradictory error status. It decodes JSON-object strings but does not infer failure from arbitrary natural-language text, nested fields or an unobserved post-state. It finds zero instances under that definition. Nine RAC compensation-event records are marked successful without an independent state-restoration observation. This count is restricted to records labelled `react_agent_compensation`; the other frameworks' repeated progress records are not pooled into it. Thus the archive neither corroborates the constructed mismatch's occurrence nor proves that all compensations achieved their intended effects.

The archive is also insufficient for an independent reproduction of every reported domain and evaluator. The linked anonymous task artifact was expired when accessed. We retain these access and provenance limits rather than imputing successful outcomes from token counts or choosing the best duplicate record. No published success-rate claim is revised by our local probes.

7. Related work and limits

RAC is the subject of the audit and already studies compensation in agent workflows. [Verified Tool Calls](#) is direct prior work on validating tool effects. [Where Agent Frameworks Fall Short](#) already studies interface compatibility and explicitly motivates version-aware API-sequence testing. The [Agent Crash Test](#) artifact likewise includes controlled tool faults and effect oracles. Our controlled failures do not establish novelty for fault injection, typed errors, postcondition verification or framework bug taxonomies.

The evidence remains concentrated in a small set of chosen interfaces. The two independent implementations add narrowly selected coordinator and transport controls, not a representative sample of recovery systems or matched end-to-end tasks. The operations are constructed, the declarations and failure modes are selected, and the experiments contain no autonomous model decisions. Native MCP airline controls add coupled business state but only two chosen booking profiles. The separate 2,000-record retry census covers the full public initial reservation population, whose ledger structures are homogeneous; it adds no autonomous decisions or framework comparison. The retail extension covers a second native environment and every eligible initial order/payment alternative for one source-selected composition. It does not make the sequence representative of actual agent behavior or the pair observations independent. These status/ledger contracts are narrower than full restoration, and no official airline or retail task success rate is reported. A state oracle is accessible by construction. These conditions make the mechanisms inspectable but limit deployment claims.

8. Offline evidence and reproducibility scope

The accompanying offline artifact contains the immutable reports, selected raw state and WAL files, the public initial airline database, the original published RAC archive, the frozen retry protocol and inspection copies of the experiment scripts. A manifest records every member's byte length and SHA-256. The separate archive receipt pins the whole package. Original machine paths in records are preserved and mapped to packaged members; reanalysis does not require access to the original research machine.

A standard-library-only Python entry point recomputes the 25-case airline state comparison, every transition and signed ledger net of the 2,000-record retry census, and the released archive's duplication, prefix-chain and structured-status results. The current package also verifies the retail source and result archives, independently reconstructs the 543-path selection and recomputes every affected state and ledger from its 663 calls. It reads saved data rather than importing the native tool or the original measurement functions. It also rejects missing, extra or altered package members. The README states exactly which other probe reports are provided only for byte-verified inspection. Successful offline verification does not imply native framework reexecution, original model-benchmark reproduction, or independent human adjudication.

Full reexecution requires the separately pinned upstream sources and runtime dependencies. Several original experimental launchers contain explicit Windows interpreter paths, which must be adapted to another machine. We therefore distinguish portable offline reanalysis from an environment-portable end-to-end launcher. The artifact adds reproducibility support for the reported diagnostic observations; it adds no new experimental observations. The retained v1/v2 bundles predate section 5.8. The v3 bundle explicitly adds the pinned retail source package, raw archive, protocol and integrated reanalysis; the older packages' scope is unchanged. Complete final retail database restoration is represented by the runtime assertion and equal recorded hashes, rather than an archived full final database.

9. Provisional conclusion

The executed probes demonstrate specific inconsistencies between returned tool errors, automatic compensation discovery and recovery records. They also show why recording the complete dependency combination matters: an adapter's error-delivery default changes the same constructed workflow's recovery result. The independent transport control shows that explicit MCP errors can be propagated consistently with recovery status; the no-op control separates that property from verified restoration. Native airline controls extend the distinction to a coupled reservation/payment ledger: equal successful payloads can accompany different persisted states, and exceptions can follow completed cancellations. The native retry census further separates a preserved status/net contract from exact state idempotence, without inferring financial harm from ledger growth. The retail composition census additionally finds normally returning cancellations with an incorrect per-method ledger, despite every direct-cancellation control meeting the declared contract. The archival analysis prevents these findings from being misrepresented as observed benchmark failures. Broader realistic workflows and submission-quality positioning remain necessary before this working manuscript can support a publication claim.

References

- [1] SagaOps. agent-saga. GitHub software repository Commit 4310ff570e60c42c081ae216e87a1ccb093525d4; inspected and selected paths executed 2026-09-19 [Source](#).
- [2] genglongling. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. GitHub software repository Commit 2781c33edc4005b066671d5dc3cad157fae14f11; coordinator controls executed 2026-09-19, not model-benchmark reproduction [Source](#).
- [3] Perera, Srinath; Hapuarachchi, Kaviru; Leymann, Frank; Khalaf, Rania. 2026. Robust Agent Compensation (RAC): Teaching AI Agents to Compensate. [Source](#).
- [4] Mansoor, Isham Kalappurackal; Phadke, Abhishek; Rana, Pratip. 2026. Verified Tool Calls Improve LLM Agent Reliability Under Non-Atomic Failures. [Source](#).
- [5] Zhu, Xinxue; Mu, Yanzhou; Zhang, Xiaoyu; Li, Tianlin; Shen, Chao; Fang, Chunrong; Liu, Yang; Zhai, Juan. 2026. Where Agent Frameworks Fall Short: Examining Functional Challenges and Usability Concerns. Version 4 [Source](#).
- [6] pavloparaschakis. Agent Crash Test. GitHub software repository README inspected 2026-09-19; implementation not executed [Source](#).
- [7] LangChain. 2026. LangChain MCP Adapters. PyPI software release 0.3.2 [Source](#).
- [8] Hapuarachchi, Kaviru. 2026. Kavirubc/react-agent-compensation: ACMCAIS26-AE-1. Zenodo [Source](#).
- [9] Sierra. 2026. tau2-bench: Native airline and retail tools, databases and fixtures. GitHub software repository Commit b7ea9074c1cba482b30687fecdb5c8425fd6f619; native tools executed on selected diagnostic fixtures, not an official task evaluation [Source](#).